REFERENCE

Checkpointing

Track, rewind, and summarize Claude's edits and conversation to manage session state.

Claude Code automatically tracks Claude's file edits as you work, allowing you to quickly undo changes and rewind to previous states if anything gets off track.

How checkpoints work

As you work with Claude, checkpointing automatically captures the state of your code before each edit. This safety net lets you pursue ambitious, wide-scale tasks knowing you can always return to a prior code state.

Automatic tracking

Claude Code tracks all changes made by its file editing tools:

- Every user prompt creates a new checkpoint
- Checkpoints persist across sessions, so you can access them in resumed conversations
- Automatically cleaned up along with sessions after 30 days (configurable)

Rewind and summarize

Run `/rewind`, or press `Esc` twice when the prompt input is empty, to open the rewind menu.

> If the prompt input contains text, double `Esc` clears it instead of opening the menu. The cleared text is saved to your input history, so press `Up` to recall it after you finish in the rewind menu.

The rewind menu lists each prompt you sent during the session. Select the point you want to act on, then choose an action:

- **Restore code and conversation**: revert both code and conversation to that point

- **Restore conversation**: rewind to that message while keeping current code
- **Restore code**: revert file changes while keeping the conversation
- **Summarize from here**: compress the conversation from this point forward into a summary, freeing context window space
- **Summarize up to here**: compress the conversation before this point into a summary, keeping later messages intact
- **Never mind**: return to the message list without making changes

After restoring the conversation or choosing Summarize from here, the original prompt from the selected message is restored into the input field so you can re-send or edit it.

Choosing Summarize up to here leaves you at the end of the conversation with the input empty.

Rewind past a cleared conversation

If you ran `/clear` earlier in the same Claude Code process, the rewind menu shows an additional entry at the top of the list labeled `/resume <session-id> (previous session)`. Select it to resume the conversation that was active before `/clear` ran. The entry is available until you exit Claude Code or resume a different session, and requires Claude Code v2.1.191 or later. On earlier versions, run `/resume` and pick the previous session from the list instead.

Restore vs. summarize

The restore options revert state: they undo code changes, conversation history, or both. The summarize options compress part of the conversation into an AI-generated summary without changing files on disk:

- **Summarize from here**: messages before the selected message stay intact. The selected message and everything after it are replaced with a summary. Use this to discard a side discussion while keeping early context in full detail.
- **Summarize up to here**: messages before the selected message are replaced with a summary. The selected message and everything after it stay intact, and you remain at the end of the conversation. Use this to compress early setup discussion while keeping recent work in full detail.

In both cases the original messages are preserved in the session transcript, so Claude can reference the details if needed. You can type optional instructions to guide what the summary focuses on. This is similar to `/compact`, but targeted: instead of summarizing the entire conversation, you choose which side of the selected message to compress.

Summarize keeps you in the same session and compresses context. If you want to branch off and try a different approach while preserving the original session intact, use **fork** instead (`claude --continue --fork-session`).

Common use cases

Checkpoints are particularly useful when:

- **Exploring alternatives**: try different implementation approaches without losing your starting point
- **Recovering from mistakes**: quickly undo changes that introduced bugs or broke functionality
- **Iterating on features**: experiment with variations knowing you can revert to working states
- **Freeing context space**: summarize a verbose debugging session from the midpoint forward, keeping your initial instructions intact

Limitations

Bash command changes not tracked

Checkpointing does not track files modified by bash commands. For example, if Claude Code runs:

```
rm file.txt
mv old.txt new.txt
cp source.txt dest.txt
```

These file modifications cannot be undone through rewind. Only direct file edits made through Claude's file editing tools are tracked.

External changes not tracked

Checkpointing only tracks files that have been edited within the current session. Manual changes you make to files outside of Claude Code and edits from other concurrent sessions are normally not captured, unless they happen to modify the same files as the current session.

Not a replacement for version control

Checkpoints are designed for quick, session-level recovery. For permanent version history and collaboration:

- Continue using version control (ex. Git) for commits, branches, and long-term history
- Checkpoints complement but don't replace proper version control
- Think of checkpoints as "local undo" and Git as "permanent history"

See also

- **Interactive mode** - Keyboard shortcuts and session controls
- **Commands** - Accessing checkpoints using `/rewind`
- **CLI reference** - Command-line options